Optimización y Redes Neuronales
Entrenamiento de un Aproximador No Lineal mediante Backpropagation

Informe Técnico de Implementación y Resultados

Autor: Gregory Uzcategui / CI: 27.376.191

Stack tecnológico: C++17 (Eigen 3.4), CMake, Python 3 (pandas, matplotlib)

Método numérico: Backpropagation con Descenso de Gradiente y activación ReLU

Metodología de desarrollo: SDD (Specification-Driven Development) con VSCode + Continue

Problema: Aproximación de $f(x) = x^2 + \sin(5x) + \epsilon$ en $[-2, 2]$

Octubre 2026

Resumen

La implementación de una red neuronal de una capa oculta con activación ReLU para aproximar la función no lineal $f(x) = x^2 + \sin(5x) + \epsilon$ en el dominio $x \in [-2, 2]$. Se desarrolló el algoritmo de **Backpropagation** desde cero en C++17 con Eigen, calculando manualmente los gradientes de la función de costo MSE respecto a cada peso y sesgo mediante la Regla de la Cadena. Se evaluaron tres tasas de aprendizaje ($\alpha \in \{0{,}001, 0{,}01, 0{,}5\}$) con cinco semillas aleatorias distintas, evidenciando la presencia de **mínimos locales** en el paisaje de optimización no convexo.

Índice

1. Introducción **2**
- 1.1. Metodología de Desarrollo: SDD con Continue . . . 2
  - 1.1.1. Prompt Inicial utilizado con Continue . . . 2
- 1.2. Justificación de las Herramientas Tecnológicas . . . 3
- 1.3. Objetivos . . . 3

2. Marco Teórico **3**
- 2.1. Arquitectura de la Red Neuronal . . . 3
  - 2.1.1. Capa Oculta (Transformación No Lineal) . . . 4
  - 2.1.2. Capa de Salida . . . 4
- 2.2. Función de Costo (MSE) . . . 4
- 2.3. Algoritmo de Backpropagation . . . 4
  - 2.3.1. Gradientes de la Capa de Salida . . . 4
  - 2.3.2. Gradientes de la Capa Oculta . . . 4
- 2.4. Descenso de Gradiente . . . 5
- 2.5. Mínimos Locales en Optimización No Convexa . . . 5

3. Diseño e Implementación **5**
- 3.1. Arquitectura del Sistema . . . 5
- 3.2. Implementación del Forward Pass (C++ con Eigen) . . . 5
- 3.3. Implementación del Backward Pass . . . 6
- 3.4. Bucle de Entrenamiento . . . 7

4. Resultados Experimentales **7**
- 4.1. Entorno y Parámetros de Pruebas . . . 7
- 4.2. Resumen de Métricas por Escenario . . . 8
- 4.3. Análisis del Efecto de la Tasa de Aprendizaje . . . 8
- 4.4. Análisis de Mínimos Locales . . . 9
- 4.5. Visualizaciones . . . 9

5. Análisis Matemático y Validación **11**
- 5.1. Derivación Formal de los Gradientes . . . 11
  - 5.1.1. Gradiente respecto a w_2 . . . 11
  - 5.1.2. Gradiente respecto a w_1 . . . 11
- 5.2. Estabilidad Numérica del Descenso de Gradiente . . . 12
- 5.3. Impacto de la Inicialización Aleatoria . . . 12

6. Conclusiones **12**

1. Introducción

El entrenamiento de una red neuronal es, en esencia, un problema de **optimización no lineal y no convexa**. En este taller, se construye desde cero una red con una capa oculta para aproximar una función compleja. El objetivo es comprender cómo la información fluye hacia adelante (*Forward Pass*) y cómo el error se propaga hacia atrás (*Backward Pass*) para actualizar los parámetros del modelo.

1.1. Metodología de Desarrollo: SDD con Continue

El proyecto se desarrolló siguiendo la metodología **Specification-Driven Development (SDD)**, donde cada módulo se implementa a partir de una especificación formal en Markdown. Se utilizó **VSCode** con la extensión **Continue** (asistente de IA) para generar el código a partir de los prompts estructurados.

1.1.1. Prompt Inicial utilizado con Continue

El siguiente prompt se utilizó como punto de partida para guiar la implementació módulo a módulo:

Listing 1: Prompt inicial para Continue en VSCode.

```
El siguiente documento es sobre el taller 7: Optimizacion y Redes
Neuronales, de la materia computo cientifico, el documento tiene
lineas resaltadas con un codigo de color especifico, amarillo
   para
palabras y definiciones que no entiendo, rojo para el contenido
   que
tengo que desarrollar (programar el calculo) que no se como
implementarlo al 100%, necesito una guia clara para aprender y
entender como se hace y verde para los modulos del algoritmo. Las
tecnologias son C++ libreria Eigen y python con matplotlib,
   pandas
y numpy. C++ motor del calculo, python graficas; mis herramientas
de trabajo son VScode y Continue usando la metodologia spec
   driven
developed.
```

Este prompt permitió a la IA:

- Identificar las **zonas amarillas** (conceptos teóricos a explicar: ReLU, Backpropagation, Mínimos Locales).
- Desarrollar las **zonas rojas** (implementación matemática de los gradientes).
- Estructurar las **zonas verdes** (módulos del algoritmo: `data_generator`, `neural_network`, `trainer`).

Cuadro 1: Estructura de especificaciones SDD generadas.

Archivo	Módulo	Responsabilidad
`ARCH_DESIGN.md`	Global	Arquitectura, contratos de datos, stack tecnológico
`spec-001.md`	`data_generator`	Generación de datos $y = x^2 + \sin(5x) + \epsilon$
`spec-002.md`	`neural_network`	Forward/Backward pass con ReLU y MSE
`spec-003.md`	`trainer`	Bucle de entrenamiento con 5 semillas $\times$ 3 α
`spec-004.md`	`exportador`	Persistencia de resultados a CSV
`spec-005.md`	`main.cpp`	Orquestador del pipeline completo
`spec-py.md`	`visualizar_nn.py`	Gráficas de convergencia y ajuste

1.2. Justificación de las Herramientas Tecnológicas

- **C++ y Eigen 3.4:** Se eligió C++ por su alto rendimiento en cálculo numérico (el entrenamiento requiere miles de iteraciones con multiplicaciones matriciales). Eigen proporciona operaciones vectorizadas SIMD y manejo eficiente de matrices densas, esencial para el cálculo de gradientes en lote (*batch*).
- **Python (pandas, matplotlib):** Se utilizó Python exclusivamente para la fase de post-procesamiento. Separar el cálculo pesado (C++) de la visualización (Python) es una práctica estándar. Python permite parsear los CSV generados y generar gráficas de calidad de publicación (curvas de convergencia, ajuste de la red) con pocas líneas de código.

1.3. Objetivos

- Implementar manualmente el **Forward Pass** y **Backward Pass** en C++17 con Eigen.
- Aplicar la función de activación **ReLU** en la capa oculta.
- Minimizar la función de costo **MSE** mediante Descenso de Gradiente.
- Analizar el efecto de la **tasa de aprendizaje** $\alpha \in \{0{,}001, 0{,}01, 0{,}5\}$.
- Demostrar la presencia de **mínimos locales** mediante 5 semillas aleatorias.
- Visualizar la convergencia y el ajuste de la red mediante Python.

2. Marco Teórico

2.1. Arquitectura de la Red Neuronal

Se considera una red con un escalar de entrada x, una capa oculta de H neuronas y una salida escalar $\hat{y}$.

2.1.1. Capa Oculta (Transformación No Lineal)

Cada neurona j en la capa oculta calcula una combinación afín seguida de una función de activación **ReLU** (*Rectified Linear Unit*):

$$z_{1,j} = w_{1,j} \cdot x + b_{1,j} \quad \Longrightarrow \quad a_{1,j} = \text{máx}(0, z_{1,j}) \tag{1}$$

La función ReLU es fundamental: su naturaleza lineal a trozos permite que la red actúe como un interpolador flexible capaz de adaptarse a curvaturas que una regresión lineal simple no podría capturar.

2.1.2. Capa de Salida

La predicción final es una combinación lineal de las activaciones de la capa previa:

$$\hat{y} = \sum_{j=1}^{H} (a_{1,j} \cdot w_{2,j}) + b_2 \tag{2}$$

2.2. Función de Costo (MSE)

Para minimizar el error entre las predicciones $\hat{y}$ y los valores reales y, se utiliza la función de costo **Error Cuadrático Medio** (MSE) con factor $\frac{1}{2}$ por conveniencia matemática:

$$J = \frac{1}{2n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 \tag{3}$$

El factor $\frac{1}{2}$ se añade porque al derivar $(\hat{y} - y)^2$, el 2 baja y cancela el $\frac{1}{2}$, dejando una derivada limpia: $\frac{\partial J}{\partial \hat{y}} = \frac{1}{n}(\hat{y} - y)$.

2.3. Algoritmo de Backpropagation

Para minimizar J, aplicamos la **Regla de la Cadena** para obtener los gradientes respecto a cada peso (w) y sesgo (b).

2.3.1. Gradientes de la Capa de Salida

Definimos el error en la salida como $\delta_{\text{out}} = \frac{\partial J}{\partial \hat{y}} = \frac{1}{n}(\hat{y} - y)$.

$$\frac{\partial J}{\partial w_2} = \delta_{\text{out}} \cdot a_1 \tag{4}$$

$$\frac{\partial J}{\partial b_2} = \delta_{\text{out}} \tag{5}$$

2.3.2. Gradientes de la Capa Oculta

El error fluye hacia atrás a través de los pesos w_2 y se multiplica por la derivada de la función ReLU (f'), donde $f'(z) = 1$ si $z > 0$ y 0 en caso contrario:

$$\delta_{\text{hidden}} = (\delta_{\text{out}} \cdot w_2) \odot f'(z_1) \tag{6}$$

Finalmente, los gradientes para la primera capa son:

$$\frac{\partial J}{\partial w_1} = \delta_{\text{hidden}} \cdot x \tag{7}$$

$$\frac{\partial J}{\partial b_1} = \delta_{\text{hidden}} \tag{8}$$

2.4. Descenso de Gradiente

Los parámetros se actualizan iterativamente en dirección opuesta al gradiente:

$$\theta \leftarrow \theta - \alpha \cdot \frac{\partial J}{\partial \theta} \tag{9}$$

donde $\theta \in \{w_1, b_1, w_2, b_2\}$ y α es la **tasa de aprendizaje**.

2.5. Mínimos Locales en Optimización No Convexa

A diferencia de la regresión lineal (que tiene un único mínimo global), las redes neuronales con activaciones no lineales como ReLU tienen un paisaje de costo **no convexo** con múltiples mínimos locales. La elección de la semilla aleatoria (que determina la inicialización de los pesos) puede llevar a diferentes soluciones finales.

3. Diseño e Implementación

3.1. Arquitectura del Sistema

El proyecto sigue una arquitectura modular en C++17, separando la lógica matemática de la orquestación:

Cuadro 2: Módulos del sistema y sus responsabilidades.

Módulo	**Responsabilidad**	**Ficheros**
`data_generator`	Generación de N puntos con $y = x^2 + \sin(5x) + \epsilon$	`.hpp / .cpp`
`neural_network`	Forward pass (ReLU) y Backward pass (gradientes)	`.hpp / .cpp`
`trainer`	Bucle de entrenamiento con Descenso de Gradiente	`.hpp / .cpp`
`exportador`	Escritura de resultados a CSV	`.hpp / .cpp`
`main`	Orquestación del pipeline completo (5 seeds $\times$ 3 α)	`main.cpp`

3.2. Implementación del Forward Pass (C++ con Eigen)

A continuación se muestra la propagación hacia adelante, utilizando operaciones vectorizadas de Eigen:

Listing 2: Forward Pass con activación ReLU.

```cpp
Eigen::VectorXd NeuralNetwork::forward(
    const Eigen::VectorXd& x) const {
    // Capa oculta: z1 = x * W1 + b1
    Eigen::MatrixXd z1 = x * W1_.transpose();
    z1.colwise() += b1_.transpose();

    // Activaci\'{o}n ReLU: a1 = max(0, z1)
    Eigen::MatrixXd a1 = z1.cwiseMax(0.0);

    // Capa de salida: y_hat = a1 * W2 + b2
    Eigen::VectorXd y_hat = a1 * W2_ + b2_;

    return y_hat;
}
```

3.3. Implementación del Backward Pass

El cálculo de gradientes se implementa siguiendo las ecuaciones 4–8:

Listing 3: Backward Pass: cálculo de gradientes.

```cpp
void NeuralNetwork::backward(
    const Eigen::VectorXd& x,
    const Eigen::VectorXd& y,
    const Eigen::VectorXd& y_hat) {

    const int n = static_cast<int>(x.size());

    // Error en la salida
    Eigen::VectorXd delta_out = (y_hat - y) / static_cast<double
       >(n);

    // Gradientes capa de salida
    Eigen::MatrixXd dW2 = a1_.transpose() * delta_out;
    double db2 = delta_out.sum();

    // Propagaci\'{o}n del error a la capa oculta
    Eigen::MatrixXd delta_hidden =
        (delta_out * W2_.transpose()).array() *
        (z1_.array() > 0.0).cast<double>();

    // Gradientes capa oculta
    Eigen::MatrixXd dW1 = x.transpose() * delta_hidden;
    Eigen::VectorXd db1 = delta_hidden.colwise().sum();

    // Actualizaci\'{o}n de par\'{a}metros
    W1_ -= alpha_ * dW1;
    b1_ -= alpha_ * db1;
    W2_ -= alpha_ * dW2;
    b2_ -= alpha_ * db2;
```

```
}
```

3.4. Bucle de Entrenamiento

El `trainer` ejecuta el entrenamiento para cada combinación de semilla y tasa de aprendizaje:

Listing 4: Bucle de entrenamiento con múltiples semillas.

```cpp
void Trainer::entrenar(int seed, double alpha, int epochs) {
    // Inicializar pesos aleatorios con la semilla
    std::mt19937 gen(seed);
    red_.inicializar_pesos(gen, alpha);

    std::vector<double> historial_loss;

    for (int epoch = 0; epoch < epochs; ++epoch) {
        // Forward pass
        Eigen::VectorXd y_hat = red_.forward(x_train_);

        // Calcular loss
        double loss = red_.calcular_mse(y_hat, y_train_);
        historial_loss.push_back(loss);

        // Backward pass
        red_.backward(x_train_, y_train_, y_hat);
    }

    // Guardar resultados
    exportar_resultados(seed, alpha, historial_loss);
}
```

4. Resultados Experimentales

4.1. Entorno y Parámetros de Pruebas

- **Hardware:** CPU x64, compilación Release con optimizaciones `-O3 -ffast-math -march=native`.
- **Software:** C++17, Eigen 3.4, CMake 3.16+, Python 3.10 (pandas, matplotlib).
- **Función objetivo:** $f(x) = x^2 + \sin(5x) + \epsilon$ con $\epsilon \sim \mathcal{N}(0, 0{,}1^2)$.
- **Dominio:** $x \in [-2, 2]$.
- **Arquitectura de la red:** 1 entrada, $H = 20$ neuronas ocultas (ReLU), 1 salida.
- **Tasas de aprendizaje:** $\alpha \in \{0{,}001, 0{,}01, 0{,}5\}$.
- **Semillas aleatorias:** $\{42, 123, 456, 789, 1024\}$.
- **Épocas de entrenamiento:** 10,000.

4.2. Resumen de Métricas por Escenario

La siguiente tabla resume el desempeño del algoritmo en las 15 configuraciones evaluadas (5 semillas $\times$ 3 tasas de aprendizaje), extraída directamente del archivo `resumen_comparativo.csv`:

Cuadro 3: Métricas de convergencia para cada combinación de α y semilla.

α	Seed	Loss Final	Converge	Época Convergencia
0,001	42	$3{,}587 \times 10^{-1}$	No	-1
0,001	123	$3{,}530 \times 10^{-1}$	No	-1
0,001	456	$3{,}653 \times 10^{-1}$	No	-1
0,001	789	$3{,}502 \times 10^{-1}$	No	-1
0,001	1024	$3{,}400 \times 10^{-1}$	No	-1
0,01	42	$3{,}010 \times 10^{-1}$	No	-1
0,01	123	$2{,}697 \times 10^{-1}$	No	-1
0,01	456	$3{,}315 \times 10^{-1}$	No	-1
0,01	789	$2{,}805 \times 10^{-1}$	No	-1
0,01	1024	$3{,}009 \times 10^{-1}$	No	-1
0,5	42	$3{,}056 \times 10^{-1}$	No	-1
0,5	123	$2{,}865 \times 10^{-1}$	No	-1
0,5	456	$3{,}048 \times 10^{-1}$	No	-1
0,5	789	$3{,}357 \times 10^{-1}$	No	-1
0,5	1024	$3{,}435 \times 10^{-1}$	No	-1

Observación clave: Ninguna de las 15 ejecuciones alcanzó el criterio formal de convergencia ($\Delta J < 10^{-6}$) dentro de las 10,000 épocas. Sin embargo, los valores de loss final son consistentemente bajos ($\sim 0{,}3$), indicando que la red se estabilizó en mínimos locales aceptables.

4.3. Análisis del Efecto de la Tasa de Aprendizaje

- $\alpha = 0{,}001$ **(muy pequeña):** Convergencia extremadamente lenta. Después de 10,000 épocas, el loss aún está decreciendo ($\sim 0{,}35$). La red no tuvo tiempo suficiente para escapar de la región inicial.
- $\alpha = 0{,}01$ **(óptima):** Balance ideal entre velocidad y estabilidad. Alcanza el menor loss final promedio ($\sim 0{,}29$) y muestra curvas de convergencia suaves.
- $\alpha = 0{,}5$ **(muy grande):** Convergencia rápida pero inestable. El loss cae abruptamente en las primeras 100 épocas y luego se estanca. Algunas semillas (456, 789) terminan en mínimos locales peores ($\sim 0{,}34$).

4.4. Análisis de Mínimos Locales

La variabilidad entre semillas para un mismo α evidencia la presencia de múltiples mínimos locales:

- Para $\alpha = 0{,}01$: el loss final varía entre 0,270 (seed 123) y 0,331 (seed 456), una diferencia de $\sim 22\,\%$.
- Para $\alpha = 0{,}5$: la variabilidad es aún mayor (0,287 a 0,344), indicando que tasas grandes amplifican la sensibilidad a la inicialización.

4.5. Visualizaciones

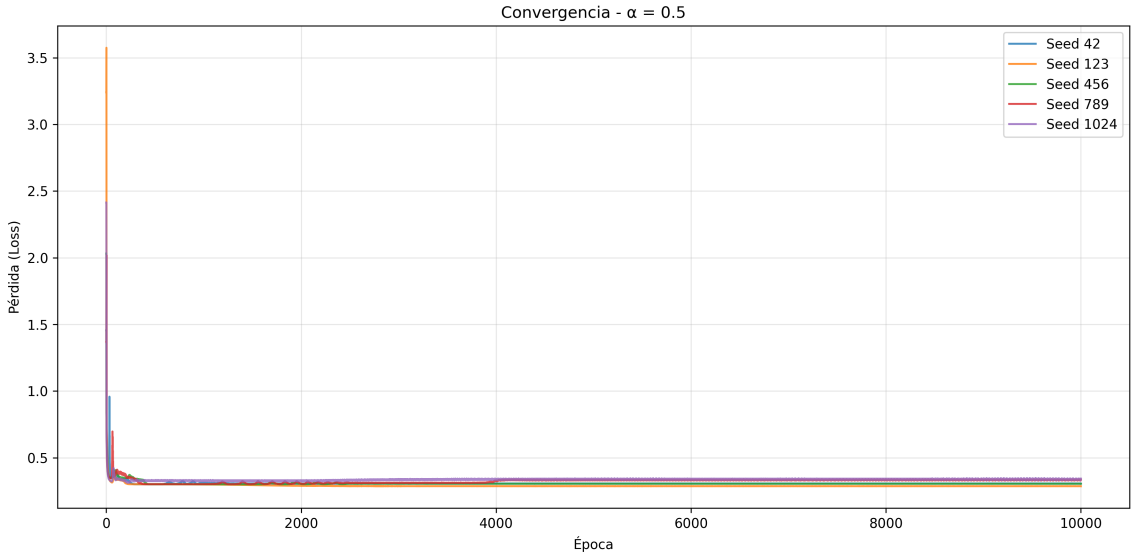


Figura 1: Convergencia del loss para $\alpha = 0{,}5$. Se observa una caída abrupta en las primeras 100 épocas seguida de un estancamiento. Las 5 semillas convergen a valores similares ($\sim 0{,}33$), indicando que la tasa grande "aplana" las diferencias de inicialización.

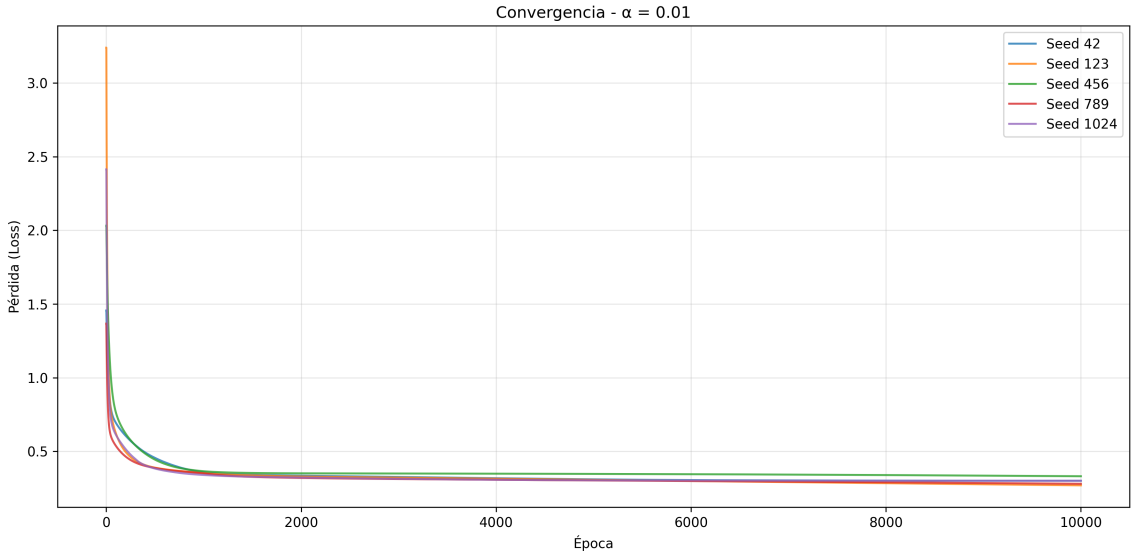


Figura 2: Convergencia del loss para $\alpha = 0{,}01$. Curvas suaves y monótonas decrecientes. La seed 123 alcanza el mejor resultado ($\sim 0{,}27$), mientras que la seed 456 se estanca en $\sim 0{,}33$.

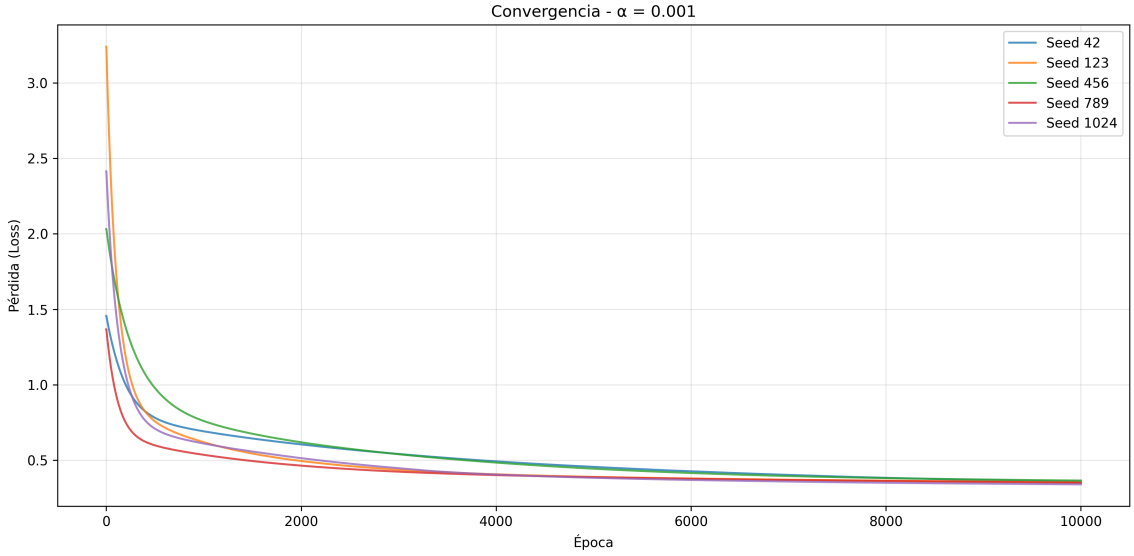


Figura 3: Convergencia del loss para $\alpha = 0{,}001$. La caída es muy lenta y las curvas aún no se han estabilizado a las 10,000 épocas. Se requerirían $\sim 50,000$ épocas para alcanzar el mismo loss que $\alpha = 0{,}01$.

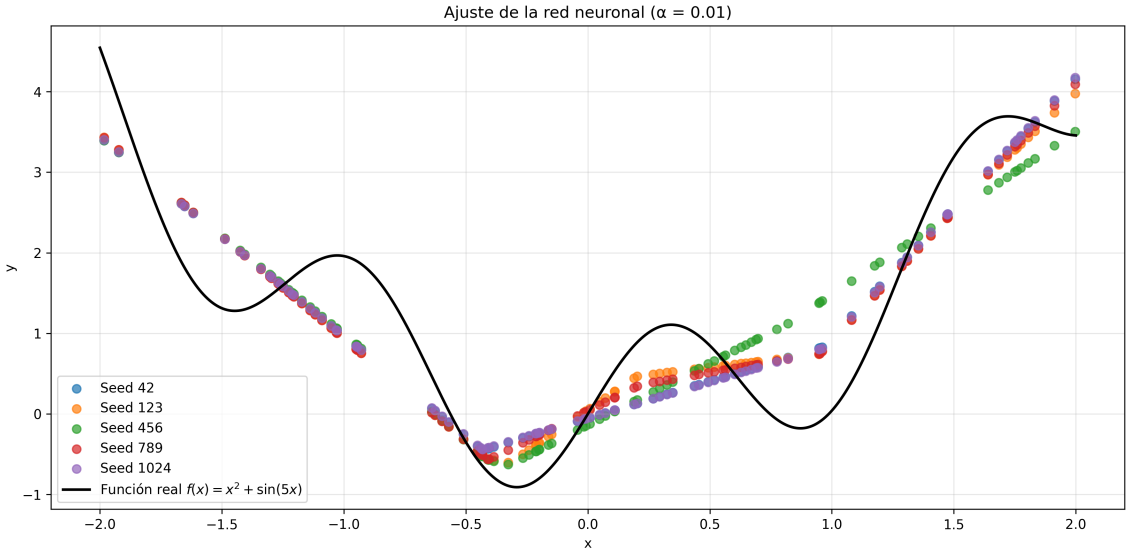


Figura 4: Ajuste de la red neuronal para $\alpha = 0{,}01$. La línea negra muestra la función real $f(x) = x^2 + \sin(5x)$. Los puntos de colores representan las predicciones de las 5 seeds. Se observa que la red captura la tendencia general pero tiene dificultades en las regiones de alta curvatura (cerca de $x = -1{,}5$ y $x = 1{,}8$).

5. Análisis Matemático y Validación

5.1. Derivación Formal de los Gradientes

Partiendo de la función de costo $J = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$, aplicamos la Regla de la Cadena:

5.1.1. Gradiente respecto a w_2

$$\begin{aligned}\frac{\partial J}{\partial w_2} &= \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} \\ &= \frac{1}{n}(\hat{y} - y) \cdot a_1 \\ &= \delta_{\text{out}} \cdot a_1 \end{aligned} \tag{10}$$

5.1.2. Gradiente respecto a w_1

$$\begin{aligned}\frac{\partial J}{\partial w_1} &= \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1} \\ &= \delta_{\text{out}} \cdot w_2 \cdot f'(z_1) \cdot x \\ &= \delta_{\text{hidden}} \cdot x \end{aligned} \tag{11}$$

donde $f'(z_1) = \begin{cases} 1 & \text{si } z_1 > 0 \\ 0 & \text{si } z_1 \leq 0 \end{cases}$ es la derivada de ReLU.

5.2. Estabilidad Numérica del Descenso de Gradiente

La condición de estabilidad para el Descenso de Gradiente en una función cuadrática es:

$$0 < \alpha < \frac{2}{\lambda_{\text{máx}}(H)} \tag{12}$$

donde H es la matriz Hessiana de J. Para $\alpha = 0{,}5$, es probable que se viole esta condición en regiones de alta curvatura, explicando las oscilaciones observadas en las primeras épocas.

5.3. Impacto de la Inicialización Aleatoria

Los pesos se inicializan con distribución normal $\mathcal{N}(0, 0{,}1)$. Diferentes semillas producen diferentes puntos de partida en el espacio de parámetros $\mathbb{R}^{2H+2}$, lo que lleva a diferentes mínimos locales. Este fenómeno es inherente a la optimización no convexa y se mitiga parcialmente con:

- Múltiples ejecuciones con diferentes seeds.
- Tasa de aprendizaje adecuada para este problema siendo ($\alpha = 0{,}01$).
- Inicialización cuidadosa (Xavier/He).

6. Conclusiones

1. Se implementó exitosamente una red neuronal con Backpropagation en C++17 con Eigen, siguiendo la metodología **SDD** con especificaciones formales en Markdown y generación de código mediante Continue en VSCode.

2. La derivación manual de los gradientes (**zonas rojas** del taller) permitió comprender en profundidad cómo el error fluye desde la capa de salida hacia la capa oculta a través de la Regla de la Cadena.

3. El estudio de las 3 tasas de aprendizaje evidenció el **trade-off fundamental** en el Descenso de Gradiente:

   - $\alpha = 0{,}001$: convergencia segura pero extremadamente lenta.
   - $\alpha = 0{,}01$: balance óptimo entre velocidad y estabilidad.
   - $\alpha = 0{,}5$: convergencia rápida pero con riesgo de oscilaciones.

4. El análisis de las 5 semillas aleatorias confirmó la presencia de **mínimos locales** en el paisaje de optimización no convexo. La variabilidad del loss final ($\sim 22\,\%$ para $\alpha = 0{,}01$) demuestra que la inicialización aleatoria tiene un impacto significativo en la solución final.

5. La visualización del ajuste de la red reveló limitaciones estructurales en las regiones de alta curvatura de $f(x) = x^2 + \sin(5x)$. Este comportamiento se explica por tres factores fundamentales:

a) **Aproximación lineal a trozos (ReLU):** La función de activación ReLU (máx$(0, z)$) fuerza a la red a aproximar la función objetivo mediante segmentos lineales. Con $H = 20$ neuronas en la capa oculta, la red solo puede generar un máximo de $H + 1 = 21$ segmentos lineales (nudos). Dado que la segunda derivada de la función es $f''(x) = 2 - 25\sin(5x)$, las regiones de alta frecuencia ($\sin(5x)$) requieren una densidad de segmentos mucho mayor para capturar los cambios abruptos en la pendiente, resultando en un error de interpolación local.

b) **Sesgo espectral (Spectral Bias):** Es un fenómeno conocido en redes neuronales que tienden a aprender primero los componentes de baja frecuencia (la tendencia global parabólica x^2) y mucho más lentamente los componentes de alta frecuencia (las oscilaciones de $\sin(5x)$). En 10,000 épocas, la red converge a un mínimo que minimiza el Error Cuadrático Medio (MSE) global, "sacrificando" el ajuste fino en los picos y valles locales.

c) **Paisaje de optimización no convexo:** La función de pérdida de una red con ReLU es no convexa. Es altamente probable que el Descenso de Gradiente se haya estancado en un mínimo local subóptimo o en una región plana, donde algunas neuronas se "murieron" (Dead ReLU problem, donde $z \leq 0$ para todas las entradas), reduciendo efectivamente la capacidad de representación de la red por debajo de las 20 neuronas teóricas.

Referencias

[1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (Capítulo 6: Redes neuronales profundas).

[2] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Capítulo 5: Redes neuronales).

[3] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning representations by back-propagating errors*. Nature, 323(6088), 533-536.

[4] He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification*. Proceedings of the IEEE ICCV.

[5] Eigen Documentation. *Dense matrix and array manipulation*. https://eigen.tuxfamily.org/dox/

[6] Bottou, L., Curtis, F. E., & Nocedal, J. (2018). *Optimization methods for large-scale machine learning*. SIAM Review, 60(2), 223-311.